

ПРАКТИЧНЕ ЗАНЯТТЯ №15

Тема: Методологія розроблення програмного продукту з використанням технологій ООП. Принципи SOLID. Закон Деметри

Звіт про виконання практичного заняття

Виконав: Вівчар Вадим Вікторович

Група: АЛК-43

Мета роботи

Ознайомитися з основними принципами SOLID, розглянути їх роль у побудові гнучкого та підтримуваного об'єктно-орієнтованого коду, а також дослідити поняття God Object, інверсію залежностей і закон Деметри на конкретних прикладах мовою C++.

Теоретичні відомості

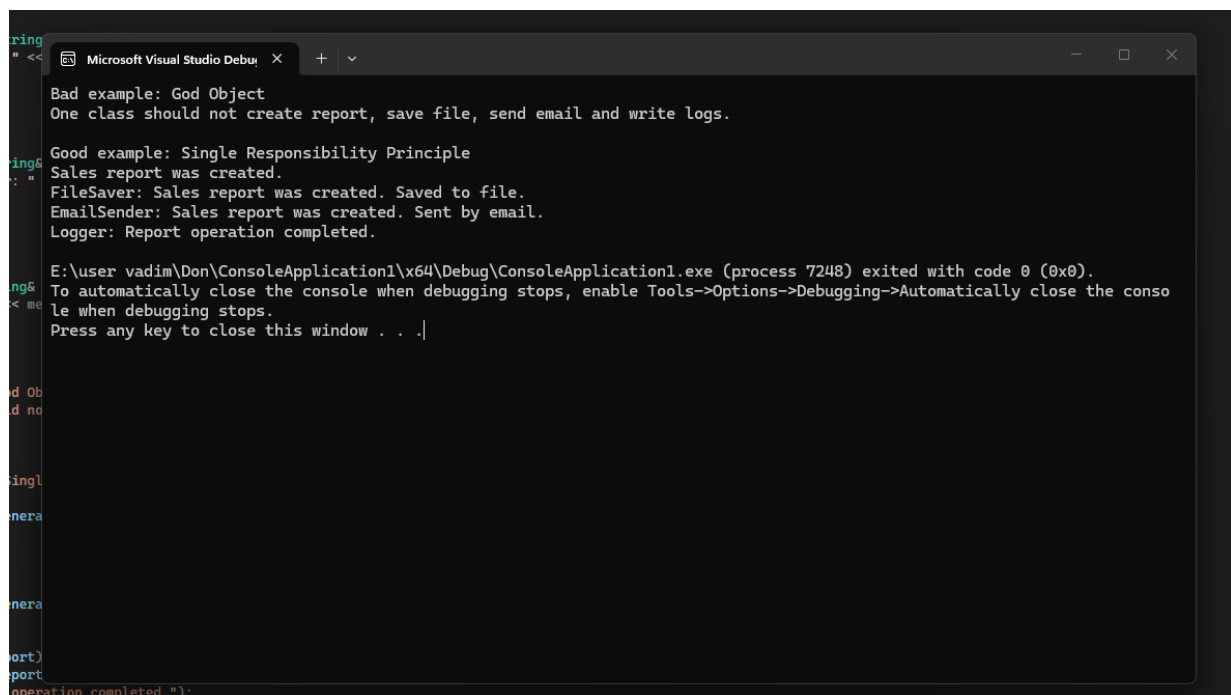
SOLID - це набір принципів об'єктно-орієнтованого проектування, які допомагають зменшити зв'язність між класами, підвищити зрозумілість коду та спростити його розширення. До SOLID належать принцип єдиної відповідальності, принцип відкритості-закритості, принцип підстановки Лісков, принцип розділення інтерфейсів і принцип інверсії залежностей.

Принцип єдиної відповідальності вимагає, щоб клас мав одну основну причину для зміни. Принцип відкритості-закритості означає, що код має бути відкритим для розширення, але закритим для зміни вже існуючої реалізації. Принцип Лісков визначає, що об'єкти похідних класів повинні коректно замінювати об'єкти базового класу.

Принцип розділення інтерфейсів рекомендує не створювати надто великі інтерфейси, які змушують класи реалізовувати зайві методи. Принцип інверсії залежностей вимагає, щоб високорівневі модулі залежали не від конкретних реалізацій, а від абстракцій. Закон Деметри доповнює ці ідеї та обмежує надмірне звертання до внутрішньої структури інших об'єктів.

Хід роботи

Під час виконання практичного заняття було створено три демонстраційні програми мовою C++. У першому прикладі показано проблему God Object і правильне розділення відповідальностей між окремими класами. У другому прикладі продемонстровано принципи OCP, LSP та ISP. У третьому прикладі показано принцип інверсії залежностей і закон Деметри.

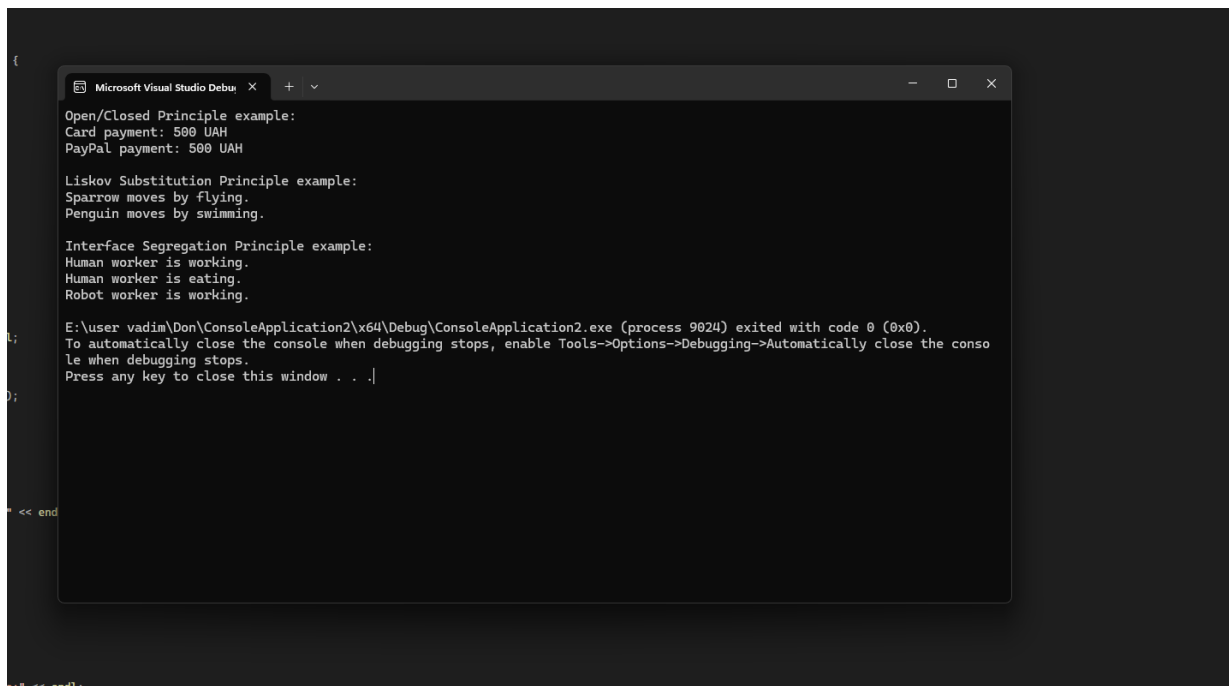


```
Microsoft Visual Studio Debug
Bad example: God Object
One class should not create report, save file, send email and write logs.

Good example: Single Responsibility Principle
Sales report was created.
FileSaver: Sales report was created. Saved to file.
EmailSender: Sales report was created. Sent by email.
Logger: Report operation completed.

E:\user vadim\Don\ConsoleApplication1\x64\Debug\ConsoleApplication1.exe (process 7248) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .|
```

Рисунок 1 - Результат демонстрації принципу єдиної відповідальності та проблеми God Object



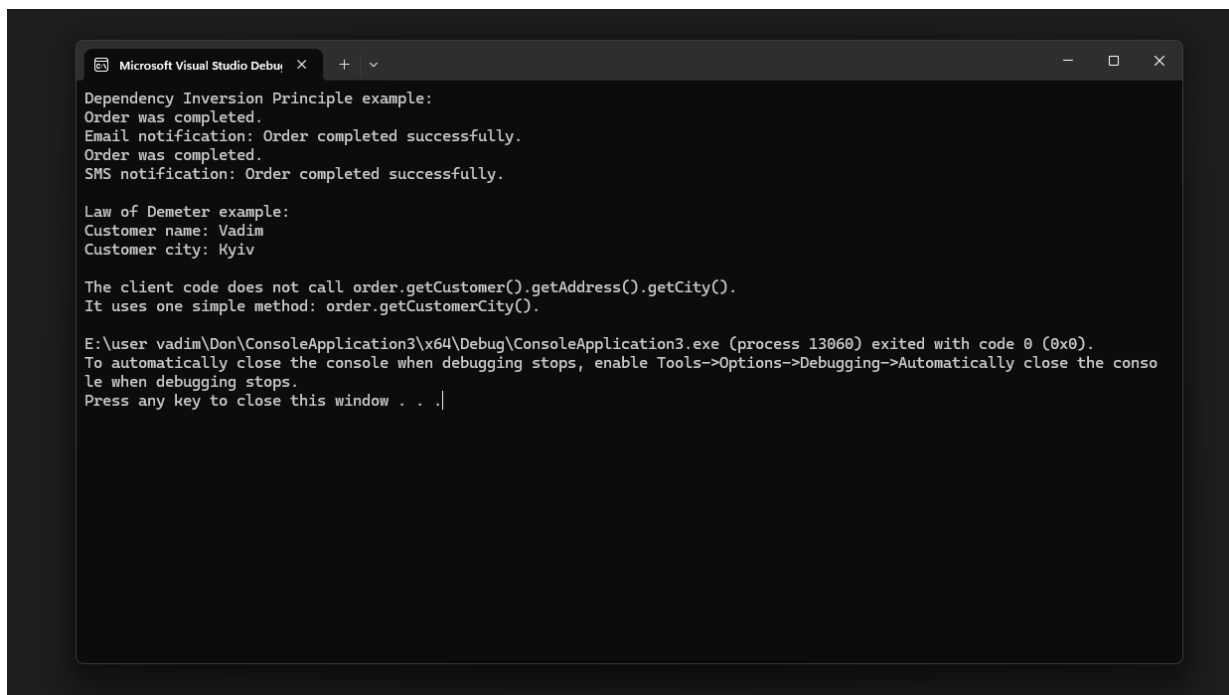
```
Microsoft Visual Studio Debu X + -
Open/Closed Principle example:
Card payment: 500 UAH
PayPal payment: 500 UAH

Liskov Substitution Principle example:
Sparrow moves by flying.
Penguin moves by swimming.

Interface Segregation Principle example:
Human worker is working.
Human worker is eating.
Robot worker is working.

E:\user vadim\Don\ConsoleApplication2\x64\Debug\ConsoleApplication2.exe (process 9024) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .|
```

Рисунок 2 - Результат демонстрації принципів OCP, LSP та ISP



```
Microsoft Visual Studio Debu X + -
Dependency Inversion Principle example:
Order was completed.
Email notification: Order completed successfully.
Order was completed.
SMS notification: Order completed successfully.

Law of Demeter example:
Customer name: Vadim
Customer city: Kyiv

The client code does not call order.getCustomer().getAddress().getCity().
It uses one simple method: order.getCustomerCity().

E:\user vadim\Don\ConsoleApplication3\x64\Debug\ConsoleApplication3.exe (process 13060) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .|
```

Рисунок 3 - Результат демонстрації принципу інверсії залежностей та закону Деметри

Контрольні запитання та відповіді

1. Що означає аббревіатура SOLID?

SOLID - це аббревіатура п'яти принципів об'єктно-орієнтованого проектування: Single Responsibility Principle, Open/Closed Principle, Liskov Substitution Principle, Interface Segregation Principle та Dependency Inversion Principle. Ці принципи допомагають створювати гнучкий, зрозумілий і зручний для підтримки код.

2. Хто і коли сформулював базовий “комплект” принципів ООП?

Базовий комплект принципів SOLID пов'язують з Робертом Мартіном, який сформулював ці принципи як рекомендації для якісного об'єктно-орієнтованого проектування. Пізніше для них почали використовувати скорочення SOLID.

3. Що означає принцип єдиної відповідальності?

Принцип єдиної відповідальності означає, що клас повинен мати одну основну причину для зміни. Якщо клас одночасно створює звіт, зберігає його у файл, надсилає поштою та записує журнал, то він має забагато обов'язків. Краще розділити ці дії між окремими класами.

4. Що таке “God Object”?

God Object - це об'єкт або клас, який знає занадто багато і виконує занадто багато різних завдань. Такий клас порушує принцип єдиної відповідальності, ускладнює тестування, підтримку і розширення програми.

5. Як принцип відкритості-закритості пов'язаний з абстракціями?

Принцип відкритості-закритості означає, що програмні сутності мають бути відкритими для розширення, але закритими для зміни. Абстракції дозволяють додавати нову поведінку через нові класи, які реалізують спільний інтерфейс, без переписування вже існуючого коду.

6. Які обмеження накладає принцип Лісков?

Принцип підстановки Лісков вимагає, щоб об'єкти похідного класу могли замінювати об'єкти базового класу без порушення логіки програми. Похідний клас не повинен послаблювати очікувану поведінку базового класу або створювати ситуації, у яких клієнтський код працює неправильно.

7. Чому інтерфейс з великою кількістю різноманітних методів краще розділити на декілька менших інтерфейсів?

Великий інтерфейс змушує класи реалізовувати методи, які їм не потрібні. Це створює зайві залежності і ускладнює код. Краще розділити такий інтерфейс на кілька менших, щоб кожен клас реалізовував лише ті методи, які справді використовує.

8. Що означає інверсія залежностей?

Інверсія залежностей означає, що високорівневі модулі не повинні напряму залежати від низькорівневих реалізацій. Обидва рівні мають залежати від абстракцій. Наприклад, сервіс замовлення повинен працювати з інтерфейсом Notifier, а не з конкретним класом EmailNotifier.

9. У чому суть закону Деметри для ООП? Поясніть правило “не більше одної крапки”.

Закон Деметри означає, що об'єкт повинен взаємодіяти лише з найближчими об'єктами і не повинен звертатися глибоко у внутрішню структуру інших класів. Правило “не більше одної крапки” пояснює це на практиці: замість ланцюжка `order.getCustomer().getAddress().getCity()` краще створити простий метод `order.getCustomerCity()`.

Висновок

У ході виконання практичного заняття було розглянуто основні принципи SOLID та закон Деметри. Практичні приклади показали, що правильне розділення відповідальностей, використання абстракцій, коректне наслідування, невеликі інтерфейси та інверсія залежностей роблять програму більш гнучкою і зручною для підтримки.

Також було встановлено, що надмірно великі класи типу God Object ускладнюють розвиток програмної системи. Дотримання принципів SOLID і закону Деметри допомагає уникати таких проблем, зменшує залежність між класами та полегшує подальше розширення програми.